Assignment 2:
Solving a Two-Echelon Capacitated Vehicle Routing Problem using Adaptive Large Neighborhood Search

1CM260 Large-scale Optimization for Transport & Mobility

Yusuf Berkan Gokce

Student ID: 2372711

Quarter 1



Eindhoven, October 12, 2026

1 Introduction

The Two-Echelon Capacitated Vehicle Routing Problem (2E-CVRP) involves distributing goods from depots to customers through intermediate facilities called satellites. Larger vehicles transport goods to satellites, while smaller vehicles deliver them to customers. The objective is to minimize travel, vehicle usage, and satellite handling costs while satisfying demand and capacity constraints.

In this assignment, we develop an Adaptive Large Neighborhood Search (ALNS) algorithm for the 2E-CVRP, extending the provided implementation with three destroy and three repair operators, adaptive operator selection, and optional Simulated Annealing (SA). We compare Random Insertion and Clarke–Wright Savings as constructive heuristics and evaluate the algorithm on 24 benchmark instances with 15 to 100 customers.

The results show that Clarke–Wright Savings consistently produces better initial solutions, reducing costs by 8.37% on average compared with Random Insertion. However, after ALNS optimization without SA, Random Insertion achieves a larger average improvement (16.31% versus 8.43%), and both initializations obtain the better final solution in 12 of the 24 instances. This suggests that the more structured Clarke–Wright solutions may be harder for the destroy and repair operators to improve, whereas Random Insertion leaves more opportunities for route restructuring. Furthermore, improvement-only acceptance outperforms SA in 32 of the 48 comparisons, indicating that accepting worse solutions does not necessarily improve performance within a limited iteration budget. Overall, the results highlight the importance of the interaction between initialization and neighborhood operators, rather than initial solution quality alone.

2 Methodology

This section presents the problem formulation, ALNS framework, constructive heuristics, destroy and repair operators, adaptive operator selection, and simulated annealing acceptance criterion.

2.1 Problem Formulation

The 2E-CVRP minimizes total distribution costs, consisting of travel distances, fixed vehicle costs, and satellite handling costs [1]. Customer demands must be satisfied without exceeding vehicle capacities, and each satellite must receive sufficient supply to serve its assigned customers.

The number of vehicles is unlimited. First-echelon vehicles have a capacity of 200 units and a fixed cost of 50, while second-echelon vehicles have a capacity of 50 units and a fixed cost of 25. Satellite handling costs are 5 per unit of demand.

We evaluate the algorithm on 24 benchmark instances with 15, 30, 50, or 100 customers, depot–satellite combinations of 2–3, 3–5, or 6–4, and categories Ca, Cb, Cc, and Cd. For example, `Ca1-2,3,15` denotes a category Ca instance with 2 depots, 3 satellites, and 15 customers. Although the datasets include time windows and service times, these are excluded from our implementation. Time-window extensions of the 2E-CVRP are studied by Dellaert et al. [2].

2.2 ALNS Framework

Adaptive Large Neighborhood Search (ALNS) improves an initial feasible solution by repeatedly removing customers using a destroy operator and reinserting them using a repair operator [3]. Each candidate solution is evaluated against the current solution using an acceptance criterion, while the best solution found is stored separately.

Our implementation extends the provided ALNS template, retaining its problem representation and overall workflow. We implemented three destroy and three repair operators, adaptive operator selection based on search performance, and an optional simulated annealing acceptance criterion. We also introduced Clarke–Wright Savings as an alternative to the original Random Insertion constructive heuristic.

2.3 Constructive Heuristics

Two constructive heuristics are used to generate initial solutions for ALNS: the Random Insertion method from the provided template and an additional Clarke–Wright Savings heuristic. The detailed pseudocode for both heuristics is provided in Appendix A.

2.3.1 Random Insertion

The original Random Insertion heuristic combines uniform random selection with greedy insertion. In the second echelon, an unserved customer is selected randomly, followed by an existing route chosen uniformly at random. If the route has enough remaining capacity, all possible insertion positions are evaluated, and the customer is placed at the position that minimizes the resulting route distance. The

first feasible route is accepted rather than comparing insertion costs across all routes. If the selected route has insufficient capacity, another route is randomly examined. When no existing route can accommodate the customer, a new route is created from a randomly selected satellite. This process continues until all customers are served.

For the first echelon, the demand at each satellite is calculated by summing the loads of its second-echelon routes. Satellites with remaining demand are then selected uniformly at random. The algorithm attempts to insert each selected satellite into an existing depot route, again using random route selection and greedy evaluation of insertion positions. The delivered quantity is limited by the vehicle's remaining capacity. If the entire demand cannot be delivered, the remaining quantity is handled in a later iteration. If the selected route is full, or no suitable insertion is made, a new route is created from a randomly selected depot.

This approach introduces randomness through customer, satellite, route, and depot selection, while using a distance-based greedy rule to determine insertion positions. Unlike roulette-wheel selection, all available candidates have equal selection probability. Customer deliveries remain unsplit, while satellite demand can be distributed across multiple first-echelon vehicles.

2.3.2 Clarke–Wright Savings

As an alternative to Random Insertion, a two-echelon adaptation of the Clarke–Wright Savings heuristic has been implemented. The heuristic was originally introduced by Clarke and Wright [4], while the adaptation follows the general approach of Hemmelmayr et al. [5]. Unlike Random Insertion, this method starts with individual routes and progressively merges them to reduce travel distance.

First, customers are assigned to satellites using distance-based roulette-wheel selection. Each satellite receives a selection probability proportional to the inverse of its distance from the customer. This means closer satellites are more likely to be selected, while more distant satellites still have a chance. After the assignments, an individual route is created for each customer, starting and ending at its assigned satellite.

For each satellite, the potential savings from connecting two customers i and j are calculated as

$$S_{ij}^{(2)} = d_{si} + d_{sj} - d_{ij}, \tag{1}$$

where s is the assigned satellite. Customer pairs are processed in descending order of savings. Two routes are merged only if they are different routes, the selected customers are at their route endpoints, and the combined load does not exceed the second-echelon capacity. Routes may also be reversed to make the connection possible. Pairs that violate these conditions are skipped, and the process continues until all candidate merges have been evaluated.

For the first echelon, the demand at each satellite is calculated from its assigned customers, and each satellite is assigned to its nearest depot. If the demand exceeds the first-echelon vehicle capacity, full-capacity deliveries are created separately. For example, a demand of 450 units results in two full deliveries of 200 units and a remaining demand of 50 units.

The remaining satellite demands are initially placed in individual depot–satellite–depot routes. The same Clarke–Wright procedure is then applied, using

$$S_{ij}^{(1)} = d_{di} + d_{dj} - d_{ij}, \tag{2}$$

where d is the common depot and i, j are satellites. Routes are merged in descending order of savings, provided they belong to the same depot, the selected satellites are route endpoints, and the combined load respects the first-echelon capacity. Full-capacity deliveries remain separate, while the remaining deliveries can be combined into routes visiting multiple satellites.

2.4 AI-Assisted Operator Design

We consulted ChatGPT to explore suitable destroy and repair operators for the 2E-CVRP, providing the problem description, capacity constraints, ALNS framework, and the reference paper by Ropke and Pisinger (2006) [3]. The AI suggested Random, Worst, Shaw Related, and Satellite/Route Removal as destroy operators, alongside Greedy, Regret-2, Regret-k, and Satellite-Aware Greedy Insertion as repair operators. It particularly recommended Shaw Related and Satellite/Route Removal combined with Regret-2 and Satellite-Aware Greedy Insertion.

After evaluating these suggestions, we selected Worst Removal for its simplicity and ability to target costly route segments, and Shaw Related Removal for its structured removal of related customers. Satellite/Route Removal was excluded due to its potentially disruptive nature and higher repair effort.

For repair, we chose Regret-2 to complement Greedy Insertion and developed Random Feasible Insertion instead of Satellite-Aware Greedy Insertion to increase search diversity. Regret-k was excluded to avoid additional computational complexity.

The operators were adapted to our implementation: Worst Removal uses marginal second-echelon distance savings, Shaw Related Removal combines distance, demand similarity, and satellite assignment, and Regret-2 evaluates alternatives from distinct routes. All repair operators share a first-echelon reconstruction procedure to restore satellite supply-demand balance.

Thus, the AI suggestions provided a starting point, while the final choices reflected computational efficiency and complementary search behavior. The complete consultation is included in Appendix B, and the implemented operators are detailed in the following subsections.

2.5 Destroy Operators

In addition to the template's Random Removal operator, Worst Removal and Shaw Related Removal have been implemented. All three operators remove customers from second-echelon routes, while first-echelon routes are reconstructed during repair to account for changes in satellite demand. Detailed pseudocode is provided in Appendix C.

2.5.1 Random Removal

Random Removal is the original destroy operator provided in the template. It selects a specified number of currently served customers uniformly at random, without replacement. Each selected customer is then removed from its second-echelon route, regardless of its location, demand, or contribution to the route cost. Since every customer has an equal probability of being selected, this operator introduces randomness into the search and can modify different parts of the solution at once. After removal, the affected routes are updated and any second-echelon routes that no longer serve customers are deleted.

2.5.2 Worst Removal

Worst Removal was added to target customers whose removal produces the largest reduction in second-echelon travel distance. For each customer i, the distance saving is calculated as

$$\Delta_i = d_{pi} + d_{in} - d_{pn}, \tag{3}$$

where p and n are the locations immediately before and after customer i in its current route. The expression measures how much shorter the route becomes when customer i is removed and its neighboring locations are connected directly.

The customer with the highest saving is removed first. Unlike Random Removal, the savings are recalculated for all remaining customers after every removal, since removing one customer changes the neighboring locations and therefore the savings of others. This process continues until the required number of customers has been removed. If multiple customers have approximately equal maximum savings, one is selected randomly.

This operator focuses on customers that contribute most to the current travel distance, giving the repair operators an opportunity to reinsert them into more suitable routes.

2.5.3 Shaw Related Removal

Shaw Related Removal was implemented to remove groups of similar customers rather than selecting them independently, following the related-removal principle discussed by Ropke and Pisinger [3]. The idea is that removing related customers together allows the repair stage to reorganize a larger, connected part of the solution.

The operator first selects a random customer as a seed and removes it. For each following removal, a reference customer is randomly selected from the customers already removed. The remaining customers are then ranked using a relatedness score:

$$R(i,j) = 0.6\frac{d_{ij}}{d_{\max}} + 0.2\frac{|q_i - q_j|}{q_{\max} - q_{\min}} + 0.2I(s_i \neq s_j), \tag{4}$$

where d_{ij} is the distance between customers, q_i and q_j are their demands, and $I(s_i \neq s_j)$ equals 1 if they are currently assigned to different satellites and 0 otherwise. The distance and demand terms are normalized to make their scales comparable; a term is set to zero if its normalization denominator is zero.

A lower relatedness score indicates more similar customers. The customer with the lowest score is removed next, with random tie-breaking when necessary. This procedure is repeated until the required number of customers has been removed.

The weights prioritize geographical proximity (0.6), while demand similarity and satellite assignment each contribute 0.2. This encourages the removal of nearby customers with similar demands, particularly those assigned to the same satellite.

2.6 Repair Operators

In addition to the template's Greedy Cheapest Insertion operator, Regret-2 Insertion and Random Feasible Insertion have been implemented. All three operators first reinsert removed customers into second-echelon routes, considering both existing routes and new routes from any satellite. Once all customers have been reinserted, first-echelon routes are reconstructed to match the updated satellite demands. Detailed pseudocode is provided in Appendix D.

2.6.1 Greedy Cheapest Insertion

Greedy Cheapest Insertion selects the customer and insertion position with the lowest additional travel distance. For every removed customer, the algorithm evaluates all positions in existing second-echelon routes that have sufficient remaining capacity. It also considers creating a new satellite–customer–satellite route from each available satellite.

For insertion between consecutive locations p and n, the additional distance of inserting customer i is

$$\Delta_i = d_{pi} + d_{in} - d_{pn}. \tag{5}$$

For a new route from satellite s, the additional distance is $2d_{si}$. The cheapest feasible option is identified for each customer, and the customer with the lowest cost among these options is inserted first. The available options are then recalculated for the remaining customers, since the routes have changed. This continues until all removed customers have been reinserted.

2.6.2 Regret-2 Insertion

Regret-2 Insertion was added to consider not only the cheapest insertion but also the cost of delaying a customer's insertion. For each removed customer, the algorithm identifies its two cheapest feasible alternatives and calculates the regret value as

$$R_i = \Delta_i^{(2)} - \Delta_i^{(1)}, \tag{6}$$

where $\Delta_i^{(1)}$ and $\Delta_i^{(2)}$ are the costs of the best and second-best alternatives.

Unlike simply comparing different positions within the same route, our implementation considers distinct route alternatives. Only the cheapest insertion position for each existing route is retained, while each possible new satellite route is treated as a separate alternative. This avoids calculating regret from two positions that belong to the same route.

Customers with only one feasible alternative are given the highest priority, since delaying their insertion could leave fewer options later. Otherwise, the customer with the largest regret is selected and inserted using its cheapest alternative. Ties are resolved using the lowest insertion cost and a fixed ordering. The process repeats until all removed customers have been inserted.

2.6.3 Random Feasible Insertion

Random Feasible Insertion was introduced to provide a more exploratory repair strategy. Instead of selecting the cheapest insertion, the algorithm randomly chooses one of the remaining removed customers and generates all feasible insertion options.

These include every capacity-feasible position in existing second-echelon routes and the possibility of creating a new route from each satellite. One option is then selected uniformly at random and applied. The process continues until no removed customers remain.

Unlike the other two repair operators, this method does not prioritize insertion cost. It can therefore produce more varied route configurations, which may help ALNS explore solutions that the greedy operators would not normally construct.

2.6.4 Shared First-Echelon Reconstruction and Feasibility

After customer reinsertion, all three repair operators use a shared procedure to reconstruct the first echelon. Satellite demands are recalculated from the second-echelon routes, and the previous first-echelon routes are discarded.

For each satellite with remaining demand, the algorithm evaluates deliveries through existing first-echelon routes or new routes from any depot. The delivery quantity is limited by the satellite's remaining demand and the vehicle's available capacity. Candidates are ranked by additional travel distance per delivered unit:

$$H = \frac{\Delta d}{q_{\text{delivered}}}. \tag{7}$$

The lowest-scoring candidate is selected, with ties resolved by higher vehicle utilization and then lower additional distance. Satellite demands are updated after each delivery until fully satisfied, allowing deliveries to be split across multiple vehicles. Finally, route costs are recalculated and feasibility is verified by checking that each customer is served exactly once, vehicle capacities are respected, and satellite supply matches second-echelon demand.

2.7 Adaptive Operator Selection

Adaptive operator selection was implemented following Ropke and Pisinger [3]. At each ALNS iteration, one destroy and one repair operator are selected independently using roulette-wheel selection, with probabilities determined by their weights:

$$P_i = \frac{w_i}{\sum_{j=1}^{m} w_j}, \tag{8}$$

where $m = 3$ for each operator group. All weights are initialized to 1, giving equal initial selection probabilities. After generating a candidate solution, both selected operators receive a reward based on the iteration outcome:

- $\omega_1 = 10$: A new global best solution is found.
- $\omega_2 = 5$: The current solution improves, but not the global best.
- $\omega_3 = 2$: A non-improving solution is accepted.
- $\omega_4 = 0$: The candidate is rejected.

The third reward applies only when a non-improving solution is accepted through simulated annealing (SA). The weights of the selected operators are updated after each iteration using exponential smoothing:

$$w_i^{\text{new}} = \lambda w_i^{\text{old}} + (1 - \lambda)\omega, \tag{9}$$

where $\lambda = 0.9$ and ω is the assigned reward. Both selected operators receive the same reward, while all other weights remain unchanged. This mechanism gradually favors operators that produce better solutions while adapting their selection probabilities throughout the search.

2.8 Simulated Annealing

Simulated Annealing (SA) was implemented as an optional acceptance criterion to allow occasional acceptance of worse solutions, potentially helping ALNS escape local optima. After each destroy–repair iteration, candidates improving the current solution are always accepted, and the global best is updated whenever improved. Worse candidates may also be accepted with probability

$$P_{\text{accept}} = \exp\left(-\frac{\Delta C}{T}\right), \qquad \Delta C = C_{\text{candidate}} - C_{\text{current}}, \tag{10}$$

where T is the current temperature. Acceptance is determined by comparing this probability with a uniformly drawn random number. Rejected candidates leave the current solution unchanged, while the global best is stored separately. The initial temperature is calibrated so that a candidate with a 5% higher cost than the initial solution has an acceptance probability of 0.5:

$$T_0 = \frac{0.05 C_0}{\ln 2}, \tag{11}$$

where C_0 is the initial solution cost. Temperature decreases geometrically after each iteration:

$$T_{k+1} = \alpha T_k, \qquad \alpha = 0.99. \tag{12}$$

As the temperature decreases, worse solutions become less likely to be accepted. SA acceptance is applied only while $T > 10^{-8}$; below this threshold, worse candidates are rejected. When SA is disabled, only improving solutions are accepted, allowing both acceptance strategies to be compared using the same ALNS operators.

3 Results

This section compares the two constructive heuristics and evaluates ALNS with and without Simulated Annealing across 24 benchmark instances, focusing on solution quality, computation time, and adaptive operator behavior.

3.1 Constructive Heuristic Comparison

The performance of Random Insertion and Clarke–Wright Savings was compared across all 24 benchmark instances. Each heuristic was executed 30 times per instance using different random seeds. The average total solution cost across these runs was used to compare their performance and reduce the influence of randomness.

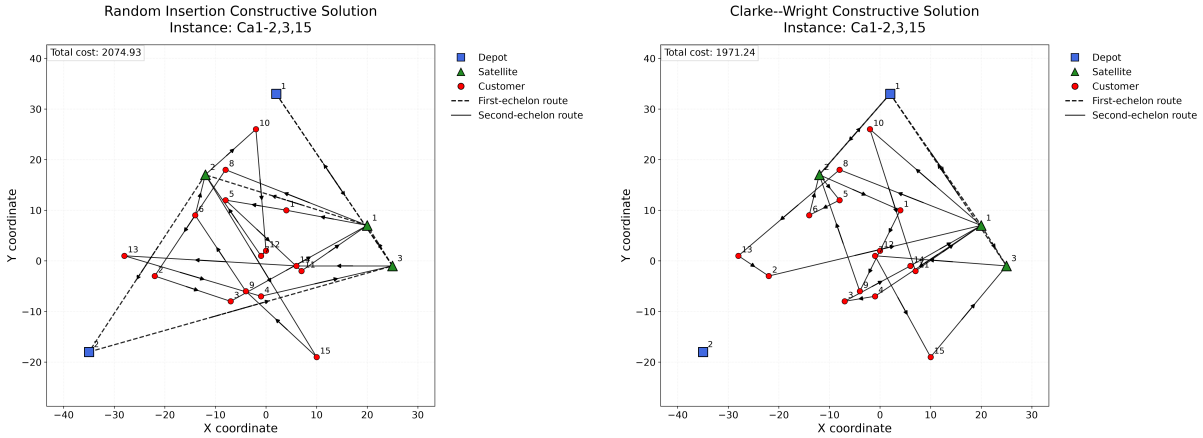
Table 1 presents the numerical results for each instance. Clarke–Wright Savings outperforms Random Insertion in all 24 instances, with an average percentage improvement of 8.37%. Figure 6 in Appendix E shows that the absolute cost difference generally increases with instance size, particularly for 50 and 100 customers.

Table 1: Average total costs of the constructive heuristics over 30 random seeds, with percentage improvement of Clarke–Wright Savings over Random Insertion.

Instance	Random Insertion	Clarke–Wright	Improvement (%)
Ca1-2,3,100	12519.80	11312.47	9.64
Ca1-2,3,15	2095.91	1972.20	5.90
Ca2-2,3,100	12949.16	11588.56	10.51
Ca3-3,5,50	5654.61	5257.74	7.02
Ca4-2,3,50	6140.89	5646.23	8.06
Ca5-6,4,50	5936.43	5364.46	9.63
Cb1-2,3,50	5953.72	5382.66	9.59
Cb2-2,3,100	11792.39	10524.65	10.75
Cb3-3,5,50	6315.18	5820.90	7.83
Cb3-6,4,100	12089.10	10774.03	10.88
Cb4-3,5,100	12479.91	11379.67	8.82
Cb5-3,5,100	12445.39	11407.35	8.34
Cc1-2,3,50	6003.96	5392.98	10.18
Cc1-6,4,50	6113.02	5505.28	9.94
Cc2-6,4,15	2136.84	2041.72	4.45
Cc3-3,5,50	5960.54	5524.90	7.31
Cc4-6,4,15	2142.15	2071.40	3.30
Cc5-6,4,100	12392.22	10978.84	11.41
Cd1-6,4,15	2060.68	2008.62	2.53
Cd2-6,4,100	11968.11	10795.85	9.79
Cd2-6,4,50	5907.21	5433.60	8.02
Cd3-6,4,50	6404.86	5882.55	8.15
Cd4-2,3,100	12074.20	10793.43	10.61
Cd5-3,5,100	12073.23	11079.07	8.23
Mean	**7983.73**	**7247.46**	**8.37**

Figure 1 illustrates the differences in route structure. Random Insertion produces several long and overlapping customer–satellite connections, whereas Clarke–Wright Savings creates more consolidated routes through savings-based merging. In this example, Clarke–Wright also avoids the more distant second depot, reducing long first-echelon connections. Consequently, the total cost decreases from 2074.93 to 1971.24 (5.00%). Although the plots represent individual runs rather than 30-run averages, they illustrate the structural differences between the two heuristics.

Although Clarke–Wright Savings consistently produces better initial solutions, this does not necessarily lead to better final ALNS results, as weaker initial solutions may offer greater opportunities for improvement. This effect is examined in the following experiments.



(a) Random Insertion (2074.93) **(b)** Clarke–Wright Savings (1971.24)

Figure 1: Comparison of initial routing solutions for instance `Ca1-2,3,15`.

3.2 ALNS Performance Without Simulated Annealing

ALNS was evaluated on all 24 benchmark instances using both constructive heuristics, with 200 iterations and random seed 42 per run. Simulated Annealing was disabled, so only improving solutions were accepted. Table 2 presents results for three representative instances with 15, 50, and 100 customers. The complete table for all 24 instances under both initializations are provided in Appendix F.

Table 2: Representative ALNS results without Simulated Annealing (200 iterations, seed 42).

Instance	Constructive	Initial cost	Final cost	Initial dist.	Final dist.	Improv. (%)	Time (s)
Ca1-2,3,100	Random	12716.99	10739.59	3666.99	1689.59	15.55	9.75
Ca1-2,3,100	CW Savings	11318.82	10710.23	2193.82	1660.23	5.38	8.43
Ca1-2,3,15	Random	2074.93	1796.13	649.93	371.13	13.44	0.31
Ca1-2,3,15	CW Savings	1971.24	1812.77	521.24	387.77	8.04	0.39
Ca3-3,5,50	Random	5543.85	4767.29	1493.85	717.29	14.01	3.15
Ca3-3,5,50	CW Savings	5186.25	4761.71	1036.25	711.71	8.19	3.02

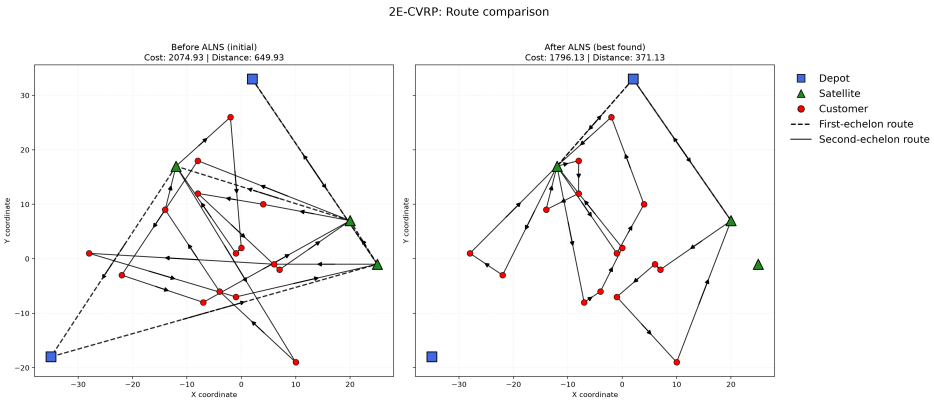
The complete results in Table 4 (Appendix F) show that ALNS reduces total costs for all instances under both initializations, ALNS reduces total costs for all instances under both initializations, with mean relative improvements of 16.31% for Random Insertion and 8.43% for Clarke–Wright Savings. The larger improvement from Random Insertion partly reflects its higher initial costs. Despite Clarke–Wright's better initial solutions, both methods achieve the lower final cost in 12 instances each, suggesting that ALNS substantially reduces the influence of initialization.

Figure 2 illustrates ALNS improvements for `Ca1-2,3,15`. Random Insertion improves from 2074.93 to 1796.13 in total cost and from 649.93 to 371.13 in traveled distance. Clarke–Wright Savings improves from 1971.24 to 1812.77 in cost and from 521.24 to 387.77 in distance. In both cases, ALNS restructures routes and customer assignments to reduce long connections. Despite starting from a worse solution, Random Insertion achieves a slightly lower final cost in this example.

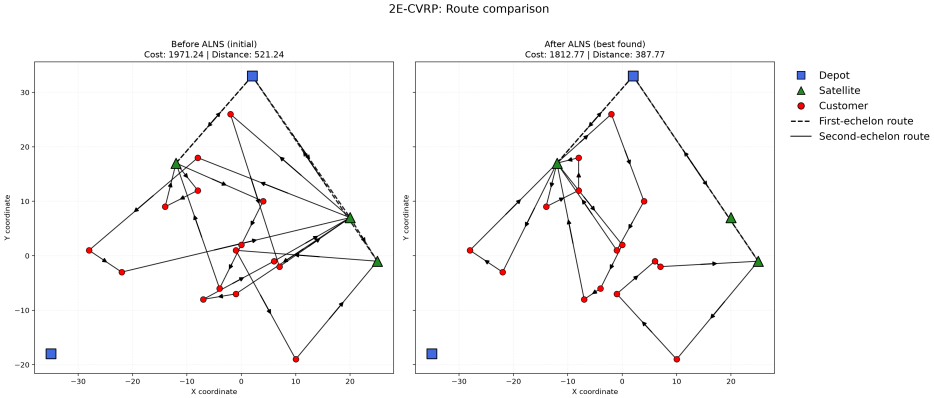
Figure 3 shows operator weight evolution during ALNS. With Random Insertion, Random and Shaw Related Removal initially receive the highest destroy weights, while Greedy and Regret-2 Insertion are favored for repair. With Clarke–Wright, Shaw Related Removal and Regret-2 receive particularly high early weights. Weights generally decline as improvements become less frequent, with occasional increases after successful iterations, demonstrating how adaptive selection responds to operator performance.

The convergence plots in Appendix G show that most improvements occur early, with the best-so-far cost generally stabilizing after approximately 25 iterations. Subsequent iterations rarely improve the solution, despite continued exploration.

Overall, ALNS substantially improves both initializations and narrows their initial cost differences. The effect of Simulated Annealing is examined in the following sections.

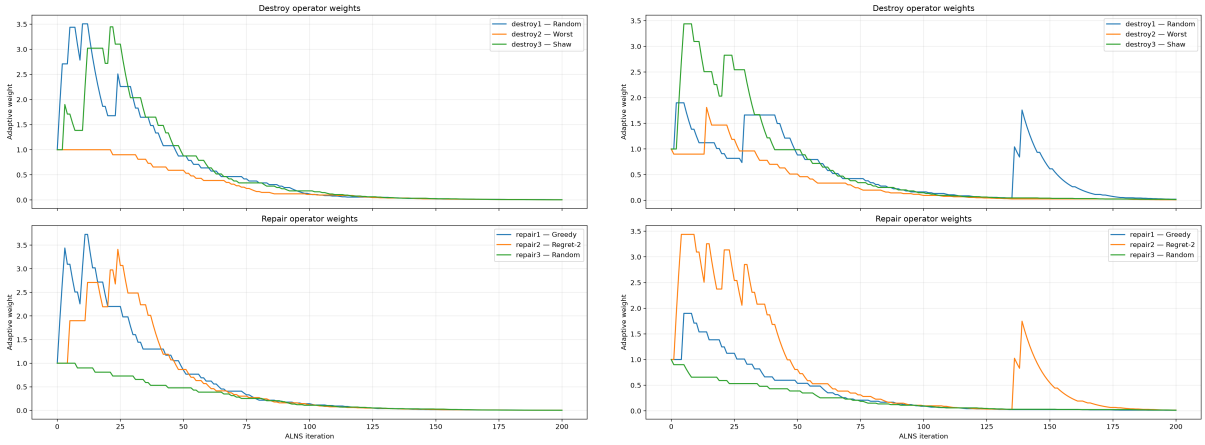


(a) Random Insertion initialization.



(b) Clarke–Wright Savings initialization.

Figure 2: Initial and best ALNS routes for `Ca1-2,3,15` using both heuristics without SA.



(a) Random Insertion initialization.

(b) Clarke–Wright Savings initialization.

Figure 3: Adaptive destroy and repair operator weights over 200 ALNS iterations for instance `Ca1-2,3,15`, without Simulated Annealing.

3.3 ALNS Performance with Simulated Annealing

The ALNS experiments were repeated with Simulated Annealing (SA) enabled. Table 3 presents results for three representative instances with 15, 50, and 100 customers. The complete results for all 24 instances under both initializations are provided in Appendix H.

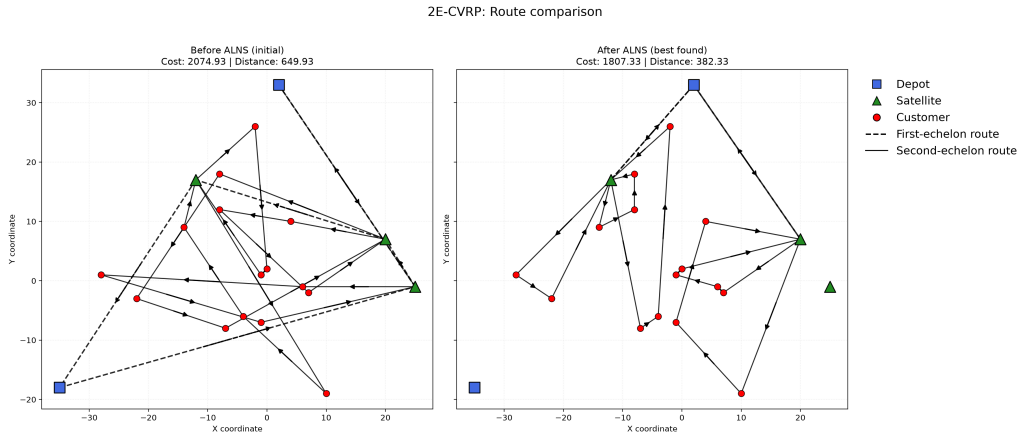
The complete results in Table 5 (Appendix H) show that, across all instances, ALNS improves every initial solution, with mean cost reductions of 16.06% for Random Insertion and 8.16% for Clarke–Wright Savings, partly reflecting Random Insertion's higher initial costs. Clarke–Wright achieves lower final

costs in 14 instances versus 10 for Random Insertion, indicating that its initial advantage narrows after ALNS.

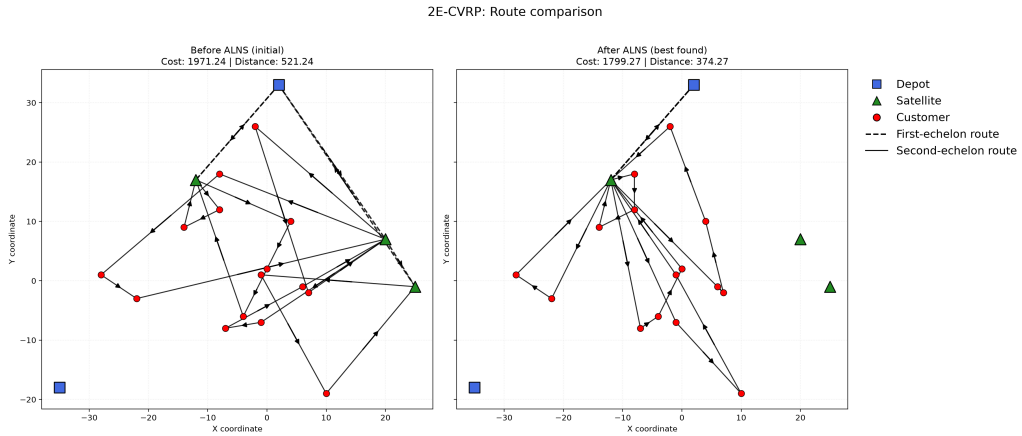
Table 3: Representative ALNS performance with Simulated Annealing (200 iterations, seed 42).

Instance	Constructive	Initial cost	Final cost	Initial dist.	Final dist.	Improv. (%)	Time (s)
Ca1-2,3,100	Random	12716.99	10777.23	3666.99	1702.23	15.25	11.99
Ca1-2,3,100	CW Savings	11318.82	10767.59	2193.82	1717.59	4.87	10.87
Ca1-2,3,15	Random	2074.93	1807.33	649.93	382.33	12.90	0.44
Ca1-2,3,15	CW Savings	1971.24	1799.27	521.24	374.27	8.72	0.42
Ca3-3,5,50	Random	5543.85	4774.57	1493.85	724.57	13.88	4.31
Ca3-3,5,50	CW Savings	5186.25	4780.96	1036.25	730.96	7.81	4.20

Figure 4 shows the route changes for `Ca1-2,3,15`. Both initializations produce shorter routes after ALNS. Random Insertion improves from 2074.93 to 1807.33, while Clarke–Wright Savings improves from 1971.24 to 1799.27. Despite its higher initial cost, Random Insertion reaches a similar final cost. Travel distances decrease from 649.93 to 382.33 and from 521.24 to 374.27, respectively.



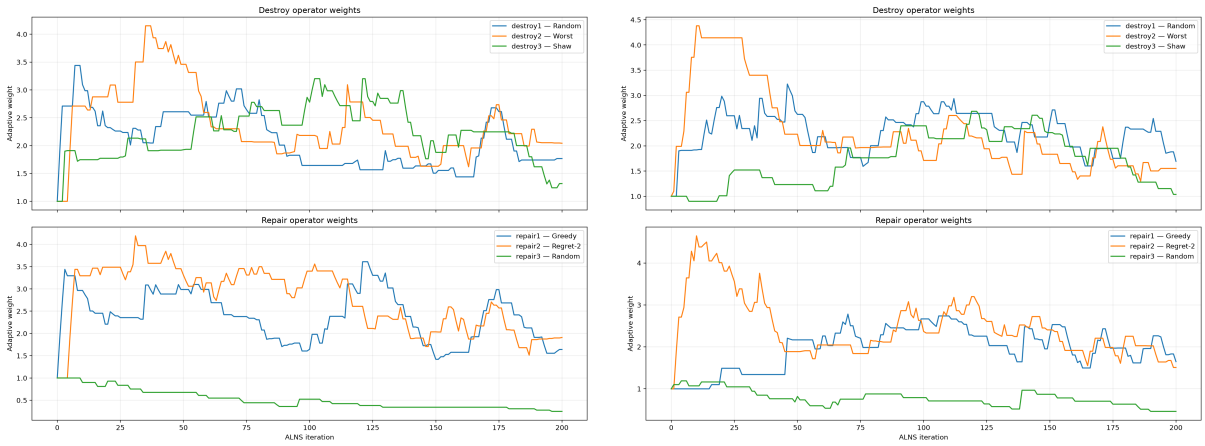
(a) Random Insertion initialization.



(b) Clarke–Wright Savings initialization.

Figure 4: Routes before and after SA-enabled ALNS for instance `Ca1-2,3,15`.

Figure 5 shows the evolution of adaptive operator weights. Worst Removal and Regret-2 receive high early rewards in both runs, while Shaw Related Removal becomes more competitive during the Clarke–Wright run. Unlike improvement-only ALNS, weights continue fluctuating rather than approaching zero, indicating continued operator rewards under SA. However, these single-run observations do not establish which operators perform best overall.



(a) Random Insertion initialization.

(b) Clarke–Wright Savings initialization.

Figure 5: Adaptive destroy and repair operator weights during SA-enabled ALNS for instance `Ca1-2,3,15`.

The convergence plots in Appendix I show that SA occasionally accepts worse solutions, raising the current cost above the best-so-far cost. Most improvements occur early, with occasional later gains. This demonstrates SA's additional exploration, although its effectiveness must be evaluated against the final costs of improvement-only ALNS.

3.4 Comparison of Acceptance Strategies

Figure 11 in Appendix J compares the percentage difference in best-found costs with and without Simulated Annealing (SA) across 24 instances and both initializations. Negative values favor SA, while positive values favor improvement-only acceptance.

Of the 48 comparisons, improvement-only ALNS achieves lower final costs in 32, SA in 15, with one tie. The mean cost difference (SA minus no-SA) is approximately 20.25, favoring improvement-only acceptance. Most percentage differences are within 1%, although some instances show larger deviations. Although SA accepts worse intermediate solutions and enables additional exploration, this does not consistently improve final costs within 200 iterations. SA also generally requires more computation time. Thus, improvement-only acceptance performs slightly better under the tested settings, although multiple random seeds are needed to assess the consistency of this advantage.

4 Conclusion

In this assignment, we developed an Adaptive Large Neighborhood Search (ALNS) algorithm for the Two-Echelon Capacitated Vehicle Routing Problem and evaluated the effects of constructive heuristics, adaptive operators, and Simulated Annealing across 24 benchmark instances.

An interesting finding is that better initial solutions do not necessarily lead to better final results. Although Clarke–Wright Savings produced consistently better initial solutions, its advantage largely disappeared after ALNS. One possible explanation is that its structured routes were already closer to a local optimum, leaving fewer opportunities for improvement. Furthermore, destroy operations may disrupt efficient route structures that repair operators cannot easily reconstruct. In contrast, Random Insertion provides less structured solutions with greater potential for route reorganization. Thus, ALNS performance depends on both initial solution quality and operator effectiveness.

The convergence and operator-weight plots provide further insight into the search behavior. Most improvements occurred early, after which finding better solutions became increasingly difficult. Adaptive weights also favored different operators throughout the search, supporting the use of multiple destroy and repair strategies.

Finally, Simulated Annealing did not consistently improve performance. Improvement-only acceptance achieved better final costs in 32 of the 48 comparisons and generally required less computation time. Thus, accepting worse solutions offers limited benefits under a restricted iteration budget.

Overall, the experiments demonstrate that ALNS can substantially reduce routing costs while reducing the influence of initial solution quality. For the tested configuration, a relatively simple improvement-only acceptance strategy provided the best balance between solution quality and computational effort. Further experiments with multiple random seeds and larger iteration budgets would help establish whether these findings generalize.

References

[1] N. Sluijk, A. M. Florio, J. Kinable, N. Dellaert, and T. Van Woensel, "Two-echelon vehicle routing problems: A literature review," *European Journal of Operational Research*, vol. 304, no. 3, pp. 865–886, 2023.

[2] N. Dellaert, T. Dashty Saridarq, T. Van Woensel, and T. G. Crainic, "Branch-and-price-based algorithms for the two-echelon vehicle routing problem with time windows," *Transportation Science*, vol. 53, no. 2, pp. 463–479, 2019.

[3] S. Ropke and D. Pisinger, "An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows," *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.

[4] G. Clarke and J. W. Wright, "Scheduling of vehicles from a central depot to a number of delivery points," *Operations Research*, vol. 12, no. 4, pp. 568–581, 1964.

[5] V. C. Hemmelmayr, J.-F. Cordeau, and T. G. Crainic, "An adaptive large neighborhood search heuristic for two-echelon vehicle routing problems arising in city logistics," *Computers & Operations Research*, vol. 39, no. 12, pp. 3215–3228, 2012.

A Constructive Heuristic Pseudocode

This appendix presents the pseudocode for the two constructive heuristics used to initialize ALNS. Both algorithms construct second-echelon routes before determining the first-echelon deliveries required at each satellite.

A.1 Random Insertion

Algorithm 1: Random Insertion Constructive Heuristic

Require: Customers C, satellites S, depots D, demands q_i, vehicle capacities Q_1, Q_2, distance matrices, random generator
Ensure: Feasible two-echelon solution (R_1, R_2)

$R_1 \leftarrow \emptyset$, $R_2 \leftarrow \emptyset$

Stage 1: Second-echelon construction
$U \leftarrow C$ ▷ Initially unserved customers
while $U \neq \emptyset$ **do**
  Select customer i uniformly at random from U
  $A \leftarrow$ copy of existing routes R_2
  inserted $\leftarrow$ **false**
  while $A \neq \emptyset$ **do**
    Select route r uniformly at random from A
    $L_r \leftarrow$ total demand served by r
    if $L_r + q_i \leq Q_2$ **then**
      Evaluate every insertion position between the satellite endpoints of r
      $p^* \leftarrow$ position minimizing the resulting route distance
      Insert customer i into route r at position p^*
      Update route load, distance, and cost
      inserted $\leftarrow$ **true**
      break
    else
      Remove r from candidate set A
    end if
  end while
  if not *inserted* **then**
    Select satellite s uniformly at random from S
    Create route $r = (s, i, s)$ with load q_i
    Add r to R_2
  end if
  $U \leftarrow U \setminus \{i\}$
end while

Stage 2: First-echelon construction
for all $s \in S$ **do**
  $d_s \leftarrow 0$
end for
for all $r \in R_2$ **do**
  $s \leftarrow$ originating satellite of route r
  $d_s \leftarrow d_s + L_r$
end for
$U_S \leftarrow \{s \in S : d_s > 0\}$
while $U_S \neq \emptyset$ **do**
  Select satellite s uniformly at random from U_S
  $a \leftarrow d_s$ ▷ Remaining satellite demand
  $A \leftarrow$ copy of existing routes R_1
  inserted $\leftarrow$ **false**
  while $A \neq \emptyset$ **do**
    Select route r uniformly at random from A

$c \leftarrow Q_1 - L_r$ ▷ Remaining capacity
if $c \leq 0$ **then**
break ▷ Full selected route
end if
if s is already visited by r **then**
Remove r from A
continue
end if
$\ell \leftarrow \min(a, c)$
Evaluate all feasible insertion positions for s
$p^* \leftarrow$ position minimizing route distance
Insert satellite s into r at p^* with delivery ℓ
Update route load, distance, and cost
inserted $\leftarrow$ **true**
break
end while
if not *inserted* **then**
Select depot d uniformly at random from D
$\ell \leftarrow \min(a, Q_1)$
Create route $r = (d, s, d)$ delivering ℓ
Add r to R_1
end if
$d_s \leftarrow d_s - \ell$
if $d_s \leq 0$ **then**
$U_S \leftarrow U_S \setminus \{s\}$
end if
end while
return (R_1, R_2)

A.2 Clarke–Wright Savings

Algorithm 2: Two-Echelon Clarke–Wright Savings Constructive Heuristic

Require: Customers C, satellites S, depots D, demands q_i, capacities Q_1, Q_2, distance matrices, random generator
Ensure: Feasible two-echelon solution (R_1, R_2)
$R_1 \leftarrow \emptyset$, $R_2 \leftarrow \emptyset$

Stage 1: Customer assignment to satellites
for all $s \in S$ **do**
$C_s \leftarrow \emptyset$
end for
for all $i \in C$ **do**
if one or more satellites have $d_{si} = 0$ **then**
Select s randomly among those satellites
else
Compute weights $w_{is} = 1/d_{si}$ for all $s \in S$
Select satellite s by roulette-wheel sampling:
$p_{is} = \dfrac{w_{is}}{\sum_{k \in S} w_{ik}}$
end if
$C_s \leftarrow C_s \cup \{i\}$
end for

Stage 2: Second-echelon Clarke–Wright
for all $s \in S$ **do**
Initialize one route (s, i, s) for each $i \in C_s$
for all unordered pairs (i, j) in C_s **do**
Compute savings:

$S_{ij}^{(2)} = d_{si} + d_{sj} - d_{ij}$
end for
Randomize equal-saving ties
Sort pairs by decreasing $S_{ij}^{(2)}$
for all pairs (i, j) in sorted order **do**
if $S_{ij}^{(2)} < 0$ **then**
continue
end if
Find current routes r_i and r_j containing customers i and j
if $r_i = r_j$ **then**
continue
end if
if i or j is not a route endpoint **then**
continue
end if
if $L_{r_i} + L_{r_j} > Q_2$ **then**
continue
end if
Reverse route sequences if needed so i is last and j is first
Merge r_i and r_j into one route
Update route load and customer sequence
end for
Add all remaining routes for satellite s to R_2
end for

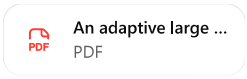
Stage 3: First-echelon Clarke–Wright

for all $s \in S$ **do**
$d_s \leftarrow$ total customer demand served from s
if $d_s = 0$ **then**
continue
end if
Assign s to its nearest depot:
$d^*(s) = \arg\min_{d \in D} d_{ds}$
Compute:
$n_s = \lfloor d_s / Q_1 \rfloor, \quad a_s = d_s \bmod Q_1$
for $k = 1$ to n_s **do**
Create route $(d^*(s), s, d^*(s))$ delivering Q_1
Add route to R_1
end for
if $a_s > 0$ **then**
Create residual route $(d^*(s), s, d^*(s))$ delivering a_s
Store route in group $G_{d^*(s)}$
end if
end for
for all depots $d \in D$ **do**
Let S_d be satellites with residual routes in G_d
for all unordered pairs (s, t) in S_d **do**
Compute savings:
$S_{st}^{(1)} = d_{ds} + d_{dt} - d_{st}$
end for
Randomize equal-saving ties
Sort pairs by decreasing $S_{st}^{(1)}$
for all pairs (s, t) in sorted order **do**
if $S_{st}^{(1)} < 0$ **then**
continue
end if
Find current residual routes r_s, r_t

if $r_s = r_t$ **then**
continue
end if
if s or t is not a route endpoint **then**
continue
end if
if $L_{r_s} + L_{r_t} > Q_1$ **then**
continue
end if
Reverse route sequences if needed so s is last and t is first
Merge r_s and r_t
Combine satellite deliveries and update load
end for
Add all remaining residual routes in G_d to R_1
end for
Validate customer coverage, capacities, and satellite supply–demand balance
return (R_1, R_2)

B AI-Assisted Operator Design

This appendix contains the original ChatGPT consultation used during the design of the additional destroy and repair operators for the 2E-CVRP.



I am working on an optimization project involving the **Two-Echelon Capacitated Vehicle Routing Problem (2E-CVRP)**, for which I am implementing an **Adaptive Large Neighborhood Search (ALNS)** algorithm.
The problem consists of two transportation levels:

- **First echelon:** Vehicles transport goods from depots to satellites.
- **Second echelon:** Vehicles deliver goods from satellites to customers.
- Vehicle capacities must be respected at both echelons, and satellite demand depends on the customers served through each satellite.
- The objective is to minimize total transportation cost across both echelons.

As part of my assignment, I need to implement at least two additional destroy operators and at least two additional repair operators for the ALNS framework.
I have attached the paper by **Ropke and Pisinger (2006)** as a reference. Please consider relevant methods from this paper, as well as other established approaches or your own ideas.
My questions are:

1. What destroy and repair operators would be suitable for a 2E-CVRP ALNS algorithm? Suggest at least 4 destroy operators and 4 repair operators.
2. For each operator, explain its underlying idea, how it works, and how it could be adapted to the two-echelon structure.
3. Discuss the advantages and disadvantages of each operator, including computational complexity, implementation difficulty, and potential impact on solution quality.
4. Which operators would complement each other well in an ALNS framework, particularly in terms of balancing diversification and intensification?
5. Based on this comparison, which 2 destroy and 2 repair operators would you recommend implementing, and why?

Please distinguish between operators directly supported by the attached paper and adaptations or additional suggestions. Focus on conceptual explanations and critical comparisons rather than code implementation.

Worked for 11s

Yes. For a 2E-CVRP, I would not simply copy standard CVRP destroy/repair operators. The useful operators are those that can change both **customer routing** and, where appropriate, **customer-to-satellite assignment**, because those second-echelon decisions determine the aggregate satellite demands that the first echelon must supply.

Ropke & Pisinger (2006) is a strong starting point. Their ALNS explicitly combines multiple removal and insertion heuristics rather than relying on one pair, because different operators work well on different instances and at different stages of the search. An adaptive large neighborhood ...

I would consider the following set.

Operator	Type	Source	Main role
Random Removal	Destroy	Ropke & Pisinger	Strong diversification
Worst Removal	Destroy	Ropke & Pisinger	Cost-focused intensification
Related/Shaw Removal	Destroy	Ropke & Pisinger, adapted	Structured neighborhood search
Satellite/Route Removal	Destroy	2E-CVRP-specific adaptation	Echelon-level diversification
Greedy Insertion	Repair	Ropke & Pisinger	Fast intensification
Regret-2 Insertion	Repair	Ropke & Pisinger	Look-ahead / robust reconstruction
Regret-3 or Regret-k	Repair	Ropke & Pisinger	Stronger look-ahead
Satellite-aware Greedy Insertion	Repair	2E-CVRP adaptation	Explicitly optimize both echelons

My recommendation, if you only need to add **two destroy + two repair operators**, would be **Related Removal + Satellite/Route Removal** and **Regret-2 + Satellite-aware Greedy Repair**. That gives you substantially different search behavior rather than four slight variations of the same idea.

1. Destroy operators

A. Random Removal — directly supported by Ropke & Pisinger

This is the simplest operator. Select q customers uniformly at random and remove them from their second-echelon routes.

Ropke & Pisinger explicitly use random removal and note that it can essentially be viewed as a particularly randomized version of Shaw removal, while being computationally cheaper.

TSP Heuristic Overview.txt

For 2E-CVRP, suppose customer i is assigned to satellite s. Removing i means:

$$\text{route containing } i \rightarrow \text{remove } i$$

and satellite demand becomes

$$D_s \leftarrow D_s - d_i.$$

The first-echelon solution may then either be immediately adjusted or reconstructed after repair, depending on your solution architecture.

Advantages: extremely easy to implement, very fast, and provides strong diversification. It does not make assumptions about what is currently "wrong" with the solution.

Disadvantages: it ignores all problem structure. It may destroy perfectly good parts of the solution while leaving problematic clusters untouched. This is therefore mainly a diversification operator rather than an intelligent improvement operator.

Approximate removal-selection complexity is only $O(q)$ if sampling is efficient.

B. Worst Removal — directly supported by Ropke & Pisinger

Ropke & Pisinger define the cost contribution of request i as essentially

$$\text{cost}(i, S) = f(S) - f(S \setminus i),$$

then preferentially remove customers with high cost contribution. They randomize selection among high-ranked candidates so that exactly the same requests are not repeatedly removed.

For a normal CVRP, a simple approximation is the saving obtained by removing i between predecessor a and successor b:

$$\Delta_i = c_{a,i} + c_{i,b} - c_{a,b}.$$

But for your **2E-CVRP**, I would make the measure more interesting:

$$\Delta_i = \Delta_i^{(2)} + \Delta_i^{(1)},$$

where $\Delta_i^{(2)}$ is the second-echelon saving and $\Delta_i^{(1)}$ estimates how reducing demand at the customer's satellite affects first-echelon cost.

That makes the operator genuinely two-echelon aware.

Advantages: directly targets expensive decisions, so it is a strong intensification operator. It can quickly fix obviously bad customer placements.

Disadvantages: it can become too greedy. The same expensive customers may repeatedly be selected, and a customer that looks expensive locally may actually support a good global structure. Randomized ranking, as in Ropke & Pisinger, helps.

Naively, evaluating all removal costs is around $O(n)$ per removal if marginal route costs are cheap, giving roughly $O(qn)$. Incremental bookkeeping can reduce this substantially.

C. Related/Shaw Removal — directly supported, but needs a 2E adaptation

This is probably the most interesting operator from the paper for your project.

Instead of removing unrelated customers, Shaw removal removes customers that are **similar** according to a relatedness measure. Ropke & Pisinger use distance, temporal information, demand/capacity characteristics, and vehicle compatibility. The motivation is that related requests are easier to reshuffle into alternative arrangements than completely unrelated requests.

For your problem, you do not necessarily need their PDPTW-related time components. Define something such as

$$R(i,j) = \alpha \frac{d_{ij}}{d_{\max}} + \beta \frac{|q_i - q_j|}{q_{\max}} + \gamma I(s_i \neq s_j),$$

where s_i is the current satellite serving customer i.

Low $R(i,j)$ means high relatedness.

You could make it even more 2E-specific by including distance to satellites:

$$R(i,j) = \alpha d_{ij} + \beta |q_i - q_j| + \gamma I(s_i \neq s_j) + \delta |d_{s_i,i} - d_{s_j,j}|.$$

The algorithm starts from one random customer and progressively removes related customers. Ropke & Pisinger deliberately randomize selection within the ranked relatedness list rather than always choosing the closest candidate. TSP Heuristic Overview.txt

This creates a "hole" in a coherent geographic/demand region of the solution, which repair can then reconstruct differently.

Advantages: excellent balance between diversification and intensification. It destroys meaningful structure without completely randomizing the solution. It can expose alternative satellite assignments and route configurations.

Disadvantages: more parameters must be tuned ($\alpha, \beta, \gamma, \ldots$). Poor normalization can make one relatedness component dominate. It is also more complicated than random/worst removal.

A straightforward implementation can approach $O(qn \log n)$ if candidates are repeatedly sorted, although Ropke & Pisinger explicitly note that full sorting can be avoided using selection techniques. TSP Heuristic Overview.txt

This would be one of my two recommended destroy operators.

D. Satellite/Route Removal — additional 2E-CVRP suggestion

This exploits something the original Ropke & Pisinger problem does not have: your explicit two-echelon hierarchy.

Select a satellite s, and then remove either:

- an entire second-echelon route originating from s, or
- some/all customers currently assigned to s.

For example:

$$s_3: \quad r_1 = (s_3, 4, 8, 12, s_3), \quad r_2 = (s_3, 6, 9, s_3)$$

could become

$$s_3: \quad r_1 = \varnothing, \quad r_2 = (s_3, 6, 9, s_3),$$

with customers $4, 8, 12$ moved to the removal pool.

The repair operator is then free to assign them back to s_3, to another route, or—if your representation permits it—to **another satellite**.

This is powerful because the current satellite assignment itself can be a local optimum. Customer-by-customer removal may struggle to cross that barrier.

Advantages: directly targets the distinctive structure of 2E-CVRP. It can substantially alter satellite loads, second-echelon route composition, and therefore first-echelon routing. Very good diversification.

Disadvantages: destructive. Removing an entire route or satellite cluster can temporarily produce a very poor partial solution, and repair becomes more expensive. It also requires careful coordination with first-echelon demand updates.

Selection itself is cheap, approximately proportional to the customers removed, but subsequent repair can be substantially more expensive.

This would be my second recommended destroy operator.

---

2. Repair operators

A. Greedy Minimum-Cost Insertion — directly supported by Ropke & Pisinger

For every removed customer i, evaluate feasible insertion positions and determine its cheapest insertion:

$$c_i = \min_{p \in P_i} \Delta f(i, p).$$

Then insert the customer having the globally smallest c_i, update the solution, and repeat.

That is exactly the basic idea of Ropke & Pisinger's greedy insertion: calculate each unplanned request's cheapest feasible insertion and insert the request with the smallest such cost.

PDF An adaptive large neighborhood ...

In your case, feasibility must include second-echelon vehicle capacity:

$$\text{load}(\text{route}) + q_i \leq Q_2.$$

If satellite reassignment is permitted, insertion candidates should be evaluated over

$$(\text{satellite}, \text{route}, \text{position}).$$

Advantages: simple, fast, deterministic, and usually good at immediately recovering a low-cost solution.

Disadvantages: myopic. Ropke & Pisinger explicitly point out that greedy insertion tends to postpone "hard" requests until later, when many good insertion possibilities have disappeared.

An adaptive large neighborhood ...

This is useful as a baseline but probably not the most interesting additional operator for your assignment.

B. Regret-2 Insertion — directly supported by Ropke & Pisinger

This addresses the main weakness of greedy insertion.

For each removed customer i, find its cheapest and second-cheapest feasible insertion costs:

$$c_i^1, \quad c_i^2.$$

Define

$$R_i = c_i^2 - c_i^1.$$

Then choose

$$i^* = \arg\max_i R_i$$

and insert i^* in its cheapest position.

The interpretation is:

> Which customer will I regret most if I do not insert it now?

This is exactly the motivation and construction given in the paper. An adaptive large neighborhood ...

Suppose:

Customer	Best	Second best	Regret
A	10	12	2
B	15	40	**25**

Customer	Best	Second best	Regret
C	8	11	3

Greedy chooses C because 8 is cheapest.

Regret-2 chooses **B**, because if its best opportunity disappears, the cost may jump from 15 to 40.

This is particularly valuable in 2E-CVRP because capacity creates exactly this kind of competition for scarce insertion opportunities.

You can treat different satellites as alternative insertion possibilities as well. Then regret reflects both route and satellite flexibility.

Advantages: much better anticipation of capacity bottlenecks than greedy insertion; usually produces robust reconstructions; still conceptually simple.

Disadvantages: more insertion costs must be evaluated and maintained. Computational cost is therefore higher than basic greedy.

This is my strongest recommendation for a repair operator.

C. Regret-3 / Regret-k — directly supported by Ropke & Pisinger

Generalize regret-2 by considering the k best alternatives.

Ropke & Pisinger use a regret-k expression based on the differences between the best insertion and the next $k-1$ alternatives. They note that larger k can identify earlier when a request has limited insertion possibilities.

For regret-3, conceptually:

$$R_i = (c_i^2 - c_i^1) + (c_i^3 - c_i^1).$$

This asks not just "is my second option bad?" but "how quickly do my alternatives deteriorate?"

Advantages: stronger look-ahead and potentially useful for tightly capacitated instances.

Disadvantages: additional computational effort, while the improvement over regret-2 may be limited. The paper's experiments actually found the insertion variants relatively close, although regret-3/regret-4 with noise performed slightly better in their tests. An adaptive large neighborhood ...

If the assignment wants two **meaningfully different** new operators, I would not choose both regret-2 and regret-3. They are too similar conceptually.

D. Satellite-Aware Greedy Insertion — additional 2E-CVRP suggestion

This is the repair operator I think is most worth developing specifically for your problem.

Standard greedy insertion asks:

$$\text{"Where can I insert customer } i \text{ cheaply?"}$$

Instead ask:

$$\text{"Which satellite, route, and position gives the smallest TOTAL 2E cost?"}$$

For candidate assignment of customer i to satellite s:

$$\Delta C(i, s, p) = \Delta C^{2nd}(i, s, p) + \Delta C^{1st}(s, q_i).$$

The first term represents the change in the second-echelon route.

The second term represents the effect of adding q_i demand to satellite s, which may alter the first-echelon routing/load.

Then select

$$(i^*, s^*, p^*) = \arg\min_{i,s,p} \Delta C(i, s, p).$$

This is fundamentally different from simply assigning the customer to the nearest satellite.

Consider:

$$d(i, s_1) = 3, \qquad d(i, s_2) = 5.$$

Locally s_1 looks better. But perhaps s_1's second-echelon vehicles are almost full and supplying additional demand there requires another first-echelon trip. Then the true changes might be

$$\Delta C(i, s_1) = 18,$$

versus

$$\Delta C(i, s_2) = 9.$$

So s_2 is globally better despite being farther away.

Advantages: explicitly optimizes the interaction between the two echelons. This is much more aligned with your actual objective than ordinary insertion.

Disadvantages: this is probably the most computationally expensive operator here because evaluating one insertion can require estimating/reoptimizing first-echelon effects. Implementation difficulty depends heavily on how your solution representation handles satellite loads and first-echelon routes.

This would be my second recommended repair operator.

---

3. Comparison

A useful high-level comparison is:

Operator	Diversification	Intensification	Speed	Implementation	2E awarenes
Random Removal	★★★★★	★	★★★★★	Easy	
Worst Removal	★★	★★★★★	★★★★	Easy–Medium	Medium if
Related Removal	★★★	★★★★	★★★	Medium	High if
Satellite/Route Removal	★★★★★	★★	★★★★ destroy / costly repair	Medium	Ve
Greedy Repair	★	★★★★★	★★★★★	Easy	Low–I
Regret-2	★★	★★★★★	★★★★	Medium	High if choices i
Regret-3/k	★★	★★★★★	★★★	Medium	
Satellite-aware Greedy	★★	★★★★★	★★–★★★	Harder	Ve

The precise complexity depends on your data structures. For repair in particular, the important computational quantity is approximately

$$\text{removed customers} \times \text{candidate satellites} \times \text{routes} \times \text{insertion positions}.$$

This is why incremental evaluation and restricting candidate satellites can become important on large instances.

---

4. Which operators complement each other?

The ALNS philosophy argues against having four operators that all do roughly the same thing. Ropke & Pisinger explicitly alternate different heuristics and adapt their selection probabilities according to recent performance. An adaptive large neighborhood ...

For **diversification**, good combinations are:

$$\boxed{\text{Satellite/Route Removal} + \text{Regret-2 Repair}}$$

because a large structural portion of the solution is destroyed, but regret repair reconstructs it carefully.

For **intensification**:

$$\boxed{\text{Related Removal} + \text{Satellite-Aware Greedy Repair}}$$

because you selectively reopen a promising local region and then reconstruct it according to the actual two-echelon objective.

Worst removal + greedy insertion is also a strong pure intensification pair:

$$\text{remove expensive decisions} \rightarrow \text{reinsert them cheaply.}$$

But it may become excessively greedy and repeatedly explore similar neighborhoods.

Random removal + satellite-aware repair is almost the opposite:

$$\text{destroy unpredictably} \rightarrow \text{rebuild intelligently.}$$

That can be useful when the algorithm has stagnated.

One result from the paper is especially relevant: their experiments found Shaw removal better than worst removal, with random removal weakest among those three in the tested configurations; they also found noise-enhanced regret-3/regret-4 slightly stronger among insertion configurations. An adaptive large neighborhood ... This does **not** prove the same ranking for 2E-CVRP, but it supports using structurally informed destruction rather than relying only on random removal.

5. My recommended 2 + 2

If you already have some basic operators and need **two additional destroy and two additional repair operators**, I would implement:

Destroy: Related Removal

It is academically well supported by Ropke & Pisinger, but you have enough freedom to create a meaningful 2E-CVRP adaptation:

$$R(i,j) = \alpha(\text{geographic similarity}) + \beta(\text{demand similarity}) + \gamma(\text{satellite relationship}).$$

It provides a good middle ground between random diversification and greedy intensification.

Destroy: Satellite/Route Removal

This is the one I would add specifically because your problem is **two-echelon**. It creates moves that standard VRP removal operators cannot easily generate and gives the search a mechanism for escaping poor satellite-level structures.

Repair: Regret-2

This is established, relatively straightforward to implement, and addresses the main weakness of greedy repair. It is particularly suitable where vehicle/satellite capacities make some customers much harder to place than others. Ropke & Pisinger's explanation directly supports this logic.

An adaptive large neighborhood ...

Repair: Satellite-Aware Greedy

This makes your ALNS explicitly optimize the coupling

$$\boxed{\text{customer assignment} \rightarrow \text{satellite demand} \rightarrow \text{first-echelon routing}}.$$

That coupling is the defining feature of your problem, so having at least one operator exploit it directly makes the algorithm much more convincing as a **2E-CVRP ALNS**, rather than a standard CVRP ALNS with an extra routing layer.

So the final portfolio I would aim for is approximately:

Diversification destroy:	Satellite/Route Removal
Balanced destroy:	Related (Shaw) Removal
Robust repair:	Regret-2 Insertion
2E-aware intensification repair:	Satellite-Aware Greedy Insertion

These four also give you a strong assignment narrative: **one literature-based structured destroy, one problem-specific structural destroy, one literature-based look-ahead repair, and one problem-specific two-echelon repair.** That is a more defensible selection than, for example, implementing random + worst removal and regret-2 + regret-3, because the latter adds less genuinely new search behavior.

One additional point from Ropke & Pisinger is worth retaining in your implementation: don't assume beforehand that the theoretically "best" operator should dominate. Their ALNS scores operators based on whether they produce new global best solutions, new improving solutions, or accepted new solutions, then adjusts their selection weights. That adaptive mechanism is exactly what allows the diversification/intensification mix above to change during the run.

Separately, your existing TSP heuristic notes describe the same broad construction principle of evaluating insertion cost $\Delta = d(i,c) + d(c,j) - d(i,j)$ and selecting a minimum-cost insertion position, which can serve as a useful conceptual bridge when implementing the second-echelon insertion-cost calculation. TSP Heuristic Overview

C Destroy Operators Pseudocode

This appendix presents the pseudocode for the three destroy operators used in ALNS. All operators remove customers from second-echelon routes and return the removed customer IDs for the subsequent repair stage.

C.1 Random Removal

Algorithm 3: Random Removal

Require: Current solution X, removal count k, random generator
Ensure: Modified solution X and removed customer set M

$C_{\text{served}} \leftarrow$ all currently served customer IDs
$k \leftarrow \min(\max(k, 0), |C_{\text{served}}|)$

Stage 1: Random customer selection
Select k distinct customers uniformly at random from C_{served}
Store selected customers in M

Stage 2: Customer removal
for all customers $i \in M$ **do**
    Find the second-echelon route r serving i
    Remove customer i from route r
    Remove its demand assignment from r
    Update the route cost
end for

Stage 3: Route cleanup
for all second-echelon routes r in X **do**
    if r serves no customers **then**
        Delete route r from X
    end if
end for
return (X, M)

C.2 Worst Removal

Algorithm 4: Worst Removal

Require: Current solution X, removal count k, distance matrix d, random generator
Ensure: Modified solution X and removed customer set M

$C_{\text{served}} \leftarrow$ all currently served customer IDs
$k \leftarrow \min(\max(k, 0), |C_{\text{served}}|)$
$M \leftarrow \emptyset$

Stage 1: Iterative worst-customer removal
while $|M| < k$ **do**
    $V \leftarrow \emptyset$ ▷ Customer–saving pairs
    for all second-echelon routes r in X **do**
        for all customers i served by r **do**
            Identify the position of customer i in r
            $p \leftarrow$ location preceding customer i
            $n \leftarrow$ location following customer i
            Calculate marginal distance saving:
            $\Delta_i = d_{pi} + d_{in} - d_{pn}$
            Add (i, Δ_i, r) to V
        end for
    end for
    if $V = \emptyset$ **then**
        break

end if
$\Delta_{\max} \leftarrow \max_{(i,\Delta_i,r)\in V} \Delta_i$
$T \leftarrow \{(i, \Delta_i, r) \in V : \Delta_i \approx \Delta_{\max}\}$
Select (i^*, Δ_{i^*}, r^*) uniformly at random from T
Remove customer i^* from route r^*
Update route cost
$M \leftarrow M \cup \{i^*\}$
▷ Savings are recalculated in the next iteration
end while

Stage 2: Route cleanup
for all second-echelon routes r in X **do**
  if r serves no customers **then**
    Delete route r from X
  end if
end for
return (X, M)

C.3 Shaw Related Removal

Algorithm 5: Shaw Related Removal

Require: Current solution X, removal count k, customer demands q_i, distances d_{ij}, satellite assignments s_i, random generator
Ensure: Modified solution X and removed customer set M
$C_{\text{served}} \leftarrow$ all currently served customer IDs
$k \leftarrow \min(\max(k, 0), |C_{\text{served}}|)$
$M \leftarrow \emptyset$
if $k = 0$ **then**
  return (X, M)
end if

Stage 1: Relatedness preparation
Record the current satellite assignment s_i for each served customer
$d_{\max} \leftarrow \max_{i,j\in C_{\text{served}}} d_{ij}$
$q_{\min} \leftarrow \min_{i\in C_{\text{served}}} q_i$
$q_{\max} \leftarrow \max_{i\in C_{\text{served}}} q_i$
$\Delta q \leftarrow q_{\max} - q_{\min}$

Stage 2: Initial random removal
Select seed customer i_0 uniformly at random from C_{served}
$M \leftarrow \{i_0\}$
$U \leftarrow C_{\text{served}} \setminus \{i_0\}$
Remove customer i_0 from its second-echelon route
Update the affected route cost

Stage 3: Related customer removal
while $|M| < k$ **do**
  Select reference customer i uniformly at random from M
  for all customers $j \in U$ **do**
    if $d_{\max} > 0$ **then**
      $D_{ij} \leftarrow d_{ij}/d_{\max}$
    else
      $D_{ij} \leftarrow 0$
    end if
    if $\Delta q > 0$ **then**
      $Q_{ij} \leftarrow |q_i - q_j|/\Delta q$
    else

$Q_{ij} \leftarrow 0$
end if
if $s_i = s_j$ **then**
$H_{ij} \leftarrow 0$
else
$H_{ij} \leftarrow 1$
end if
Calculate relatedness:
$R(i,j) = 0.60D_{ij} + 0.20Q_{ij} + 0.20H_{ij}$
end for
$R_{\min} \leftarrow \min_{j \in U} R(i,j)$
$T \leftarrow \{j \in U : R(i,j) \approx R_{\min}\}$
Select customer j^* uniformly at random from T
Remove j^* from its second-echelon route
Update the affected route cost
$M \leftarrow M \cup \{j^*\}$
$U \leftarrow U \setminus \{j^*\}$
end while

Stage 4: Route cleanup
for all second-echelon routes r in X **do**
if r serves no customers **then**
Delete route r from X
end if
end for
return (X, M)

D Repair Operator Pseudocode

This appendix presents the pseudocode for the three repair operators and the shared first-echelon reconstruction procedure. All repair operators reinsert customers into second-echelon routes before rebuilding first-echelon deliveries.

D.1 Greedy Cheapest Insertion

Algorithm 6: Greedy Cheapest Insertion

Require: Partial solution X, removed customer list M, vehicle capacity Q_2, distance matrix d
Ensure: Complete and feasible solution X

$U \leftarrow$ validated list of removed customers M
while $U \neq \emptyset$ **do**
  $B \leftarrow \emptyset$ ▷ Best option for each customer
  for all customers $i \in U$ **do**
    $O_i \leftarrow \emptyset$
    for all existing second-echelon routes r **do**
      if $L_r + q_i > Q_2$ **then**
        continue
      end if
      for all insertion positions p in r **do**
        Let u, v be the locations immediately before and after position p
        Calculate additional distance:
        $\Delta_{i,r,p} = \max(0, d_{ui} + d_{iv} - d_{uv})$
        Add option $(r, p, \Delta_{i,r,p})$ to O_i
      end for
    end for
    for all satellites $s \in S$ **do**
      if $q_i \leq Q_2$ **then**
        $\Delta_{i,s} \leftarrow 2d_{si}$
        Add new-route option $(s, \Delta_{i,s})$ to O_i
      end if
    end for
    if $O_i = \emptyset$ **then**
      error: no feasible insertion
    end if
    $o_i^* \leftarrow$ cheapest option in O_i
    Add (i, o_i^*) to B
  end for
  Select $(i^*, o^*) \in B$ with minimum insertion cost
  Break ties using customer ID and deterministic route/position ordering
  if o^* inserts into an existing route **then**
    Insert i^* at the selected position
    Update route load and cost
  else
    Create route (s, i^*, s)
    Add the new route to the second echelon
  end if
  $U \leftarrow U \setminus \{i^*\}$
end while
Rebuild first-echelon routes using Algorithm 9
Recalculate route costs and total solution cost
Validate the complete solution
return X

D.2 Regret-2 Insertion

Algorithm 7: Regret-2 Insertion

Require: Partial solution X, removed customer list M, vehicle capacity Q_2, distance matrix d
Ensure: Complete and feasible solution X

$U \leftarrow$ validated list of removed customers M
while $U \neq \emptyset$ **do**
    $B \leftarrow \emptyset$
    for all customers $i \in U$ **do**
        Generate all capacity-feasible second-echelon insertion options O_i
        Include every feasible position in existing routes
        Include one new-route option for each satellite
        $A_i \leftarrow \emptyset$
        for all existing routes r with feasible insertions **do**
            Keep only the cheapest insertion position for route r
            Add this option to A_i
        end for
        for all feasible new satellite routes **do**
            Add the new-route option to A_i
        end for
        Sort A_i by increasing insertion cost, using deterministic tie-breaking
        if $A_i = \emptyset$ **then**
            error: no feasible insertion
        end if
        $o_i^* \leftarrow A_i[1]$ ▷ Cheapest alternative
        if $|A_i| = 1$ **then**
            $g_i \leftarrow 0$ ▷ Highest-priority group
            $R_i \leftarrow 0$
        else
            $g_i \leftarrow 1$
            $R_i \leftarrow \text{cost}(A_i[2]) - \text{cost}(A_i[1])$
        end if
        Add (i, g_i, R_i, o_i^*) to B
    end for
    Select customer i^* by the following priority:
    1. Lowest priority group g_i
    2. Highest regret value R_i
    3. Lowest best-insertion cost
    4. Deterministic customer/option ordering
    Apply the cheapest option $o_{i^*}^*$
    Update the second-echelon routes
    $U \leftarrow U \setminus \{i^*\}$
end while
Rebuild first-echelon routes using Algorithm 9
Recalculate route costs and total solution cost
Validate the complete solution
return X

D.3 Random Feasible Insertion

Algorithm 8: Random Feasible Insertion

Require: Partial solution X, removed customer list M, vehicle capacity Q_2, random generator
Ensure: Complete and feasible solution X

$U \leftarrow$ validated list of removed customers M
while $U \neq \emptyset$ **do**
    Select customer i uniformly at random from U
    $O_i \leftarrow \emptyset$
    for all existing second-echelon routes r **do**
        if $L_r + q_i \leq Q_2$ **then**
            for all insertion positions p in r **do**

```
                Calculate the additional travel distance
                Add option (r, p) to O_i
            end for
        end if
    end for
    if q_i ≤ Q_2 then
        for all satellites s ∈ S do
            Add new-route option (s, i, s) to O_i
        end for
    end if
    if O_i = ∅ then
        error: no feasible insertion
    end if
    Select option o* uniformly at random from O_i
    if o* is an existing-route insertion then
        Insert customer i at the selected position
        Update the existing route
    else
        Create the selected satellite–customer route
        Add the route to the second echelon
    end if
    U ← U \ {i}
end while
Rebuild first-echelon routes using Algorithm 9
Recalculate route costs and total solution cost
Validate the complete solution
return X
```

D.4 Shared First-Echelon Reconstruction

Algorithm 9: First-Echelon Reconstruction

Require: Solution X with repaired second-echelon routes, depots D, satellites S, capacity Q_1, distance matrix d
Ensure: First-echelon routes supplying all satellite demand

```
    Stage 1: Determine satellite demand
for all satellites s ∈ S do
    a_s ← 0
end for
for all second-echelon routes r do
    s ← originating satellite of r
    a_s ← a_s + L_r
end for
Remove all previous first-echelon routes
R_1 ← ∅
U_S ← {s ∈ S : a_s > 0}

    Stage 2: Generate delivery alternatives
while U_S ≠ ∅ do
    O ← ∅
    for all satellites s ∈ U_S do
        for all existing first-echelon routes r ∈ R_1 do
            if s is already visited by r then
                continue
            end if
            c_r ← Q_1 − L_r
            ℓ ← min(a_s, c_r)
```

if $\ell \leq 0$ **then**
continue
end if
Find the position p^* in r with minimum additional distance Δd
Calculate:
$H = \Delta d/\ell, \quad u = \ell/Q_1$
Add existing-route option $(s, r, p^*, \ell, H, u, \Delta d)$ to O
end for
$\ell \leftarrow \min(a_s, Q_1)$
for all depots $d \in D$ **do**
$\Delta d \leftarrow 2d_{ds}$
$H \leftarrow \Delta d/\ell$
$u \leftarrow \ell/Q_1$
Add new-route option $(s, d, \ell, H, u, \Delta d)$ to O
end for
end for

Stage 3: Select and apply delivery
if $O = \emptyset$ **then**
error: no feasible delivery
end if
Select option $o^* \in O$ using:
1. Minimum distance per delivered unit H
2. Maximum utilization u
3. Minimum additional distance Δd
4. Deterministic satellite/route ordering
if o^* inserts into an existing route **then**
Insert satellite s^* at position p^*
Assign delivery quantity ℓ^*
Update the route load and cost
else
Create route (d^*, s^*, d^*) delivering ℓ^*
Add the route to R_1
end if
$a_{s^*} \leftarrow a_{s^*} - \ell^*$
if $a_{s^*} \leq 0$ (within numerical tolerance) **then**
$U_S \leftarrow U_S \setminus \{s^*\}$
end if
end while
return R_1

E Constructive Heuristic Cost Comparison

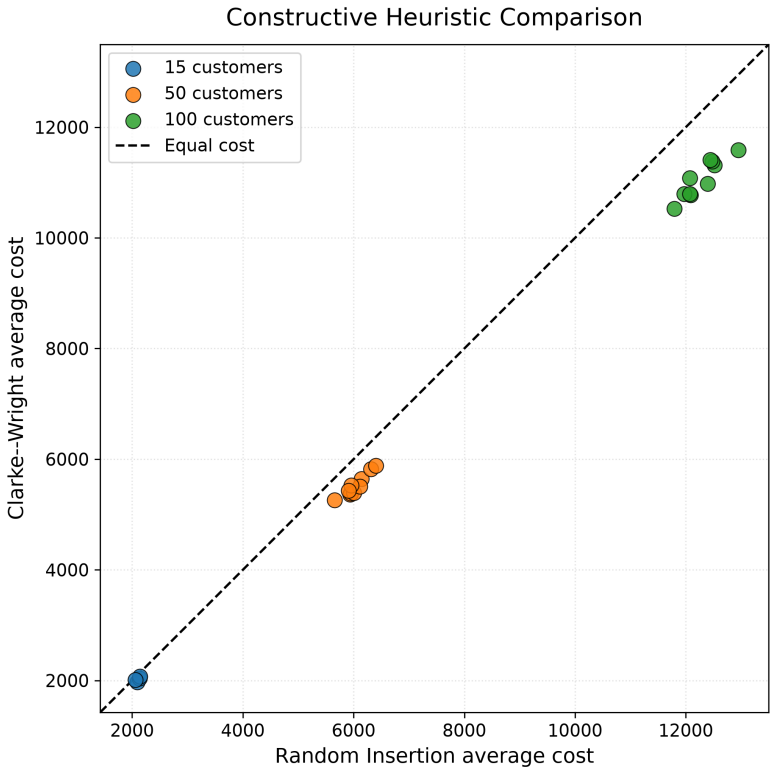


Figure 6: Average initial solution costs of Random Insertion and Clarke–Wright Savings across 30 random seeds per instance. Each point represents one instance; the dashed diagonal indicates equal costs.

All points lie below the dashed line, confirming that Clarke–Wright Savings consistently produces lower-cost initial solutions. Furthermore, as instance size increases, the points generally move further from the diagonal, indicating that the absolute cost savings become larger, particularly for instances with 50 and 100 customers. This suggests that the savings-based route merging of Clarke–Wright becomes increasingly beneficial in larger instances, where more opportunities exist to consolidate routes and reduce travel distances.

F Complete ALNS Results Without Simulated Annealing

Table 4 presents the complete ALNS performance results without Simulated Annealing for all 24 benchmark instances using both constructive heuristics. Initial and final costs, traveled distances, relative improvements, and computation times are reported for each run.

Table 4: ALNS performance without Simulated Annealing (200 iterations, seed 42).

Instance	Constructive	Initial cost	Final cost	Initial dist.	Final dist.	Improv. (%)	Time (s)
Ca1-2,3,100	Random	12716.99	10739.59	3666.99	1689.59	15.55	9.75
Ca1-2,3,100	CW Savings	11318.82	10710.23	2193.82	1660.23	5.38	8.43
Ca1-2,3,15	Random	2074.93	1796.13	649.93	371.13	13.44	0.31
Ca1-2,3,15	CW Savings	1971.24	1812.77	521.24	387.77	8.04	0.39
Ca2-2,3,100	Random	13097.88	10987.56	3847.88	1837.56	16.11	8.62
Ca2-2,3,100	CW Savings	11614.96	10943.34	2339.96	1793.34	5.78	8.55
Ca3-3,5,50	Random	5543.85	4767.29	1493.85	717.29	14.01	3.15
Ca3-3,5,50	CW Savings	5186.25	4761.71	1036.25	711.71	8.19	3.02
Ca4-2,3,50	Random	6154.25	5291.14	1704.25	891.14	14.02	2.95
Ca4-2,3,50	CW Savings	5642.64	5315.88	1217.64	915.88	5.79	2.75
Ca5-6,4,50	Random	5846.63	4872.55	1671.63	697.55	16.66	3.13
Ca5-6,4,50	CW Savings	5264.19	4952.72	989.19	727.72	5.92	2.80
Cb1-2,3,50	Random	6033.67	4903.88	1933.67	853.88	18.72	3.12
Cb1-2,3,50	CW Savings	5331.35	4896.45	1206.35	846.45	8.16	2.79
Cb2-2,3,100	Random	11961.37	9905.30	3726.37	1670.30	17.19	7.76
Cb2-2,3,100	CW Savings	10583.71	9922.05	2273.71	1687.05	6.25	8.50
Cb3-3,5,50	Random	6351.72	5316.82	1811.72	851.82	16.29	2.81
Cb3-3,5,50	CW Savings	5817.25	5307.11	1202.25	817.11	8.77	2.89
Cb3-6,4,100	Random	12130.98	10033.85	3655.98	1558.85	17.29	8.64
Cb3-6,4,100	CW Savings	10851.34	10135.97	2326.34	1660.97	6.59	7.98
Cb4-3,5,100	Random	12499.03	10532.42	3534.03	1642.42	15.73	9.55
Cb4-3,5,100	CW Savings	11465.17	10575.62	2425.17	1710.62	7.76	8.12
Cb5-3,5,100	Random	12118.71	10509.45	3233.71	1674.45	13.28	9.23
Cb5-3,5,100	CW Savings	11503.96	10561.20	2568.96	1726.20	8.20	9.35
Cc1-2,3,50	Random	6196.23	4995.55	2021.23	870.55	19.38	3.05
Cc1-2,3,50	CW Savings	5349.63	4967.46	1149.63	842.46	7.14	3.52
Cc1-6,4,50	Random	6114.40	5061.72	1839.40	711.72	17.22	3.51
Cc1-6,4,50	CW Savings	5597.36	5044.29	1147.36	694.29	9.88	3.10
Cc2-6,4,15	Random	2119.04	1837.02	694.04	362.02	13.31	0.39
Cc2-6,4,15	CW Savings	2021.91	1796.21	521.91	346.21	11.16	0.38
Cc3-3,5,50	Random	6268.65	4997.49	1893.65	722.49	20.28	2.71
Cc3-3,5,50	CW Savings	5476.91	5007.44	1101.91	732.44	8.57	3.23
Cc4-6,4,15	Random	2099.76	1777.95	599.76	277.95	15.33	0.43
Cc4-6,4,15	CW Savings	2086.50	1782.86	486.50	282.86	14.55	0.39
Cc5-6,4,100	Random	12569.12	10182.41	3969.12	1582.41	18.99	9.58
Cc5-6,4,100	CW Savings	11080.54	10067.49	2430.54	1467.49	9.14	9.35
Cd1-6,4,15	Random	2156.18	1706.09	731.18	331.09	20.87	0.37
Cd1-6,4,15	CW Savings	2116.68	1695.01	591.68	320.01	19.92	0.39
Cd2-6,4,100	Random	12036.79	10009.30	3436.79	1409.30	16.84	9.67
Cd2-6,4,100	CW Savings	10733.99	10049.38	2083.99	1449.38	6.38	9.15
Cd2-6,4,50	Random	6011.80	4993.20	1686.80	768.20	16.94	3.28
Cd2-6,4,50	CW Savings	5445.73	5000.67	1145.73	775.67	8.17	3.21
Cd3-6,4,50	Random	6496.05	5405.92	1896.05	780.92	16.78	3.26

Continued on next page

Table 4 continued from previous page

Instance	Constructive	Initial cost	Final cost	Initial dist.	Final dist.	Improv. (%)	Time (s)
Cd3-6,4,50	CW Savings	5897.50	5301.58	1197.50	751.58	10.10	3.08
Cd4-2,3,100	Random	12060.67	10333.23	3560.67	1883.23	14.32	9.53
Cd4-2,3,100	CW Savings	10728.26	10325.14	2253.26	1875.14	3.76	9.41
Cd5-3,5,100	Random	11736.62	10234.00	3086.62	1584.00	12.80	10.08
Cd5-3,5,100	CW Savings	11230.58	10250.99	2480.58	1575.99	8.72	10.16

G ALNS Convergence Without Simulated Annealing

Figures 7 and 8 show the total solution cost and traveled distance across 200 iterations for instance `Ca1-2,3,15`, using improvement-only acceptance.

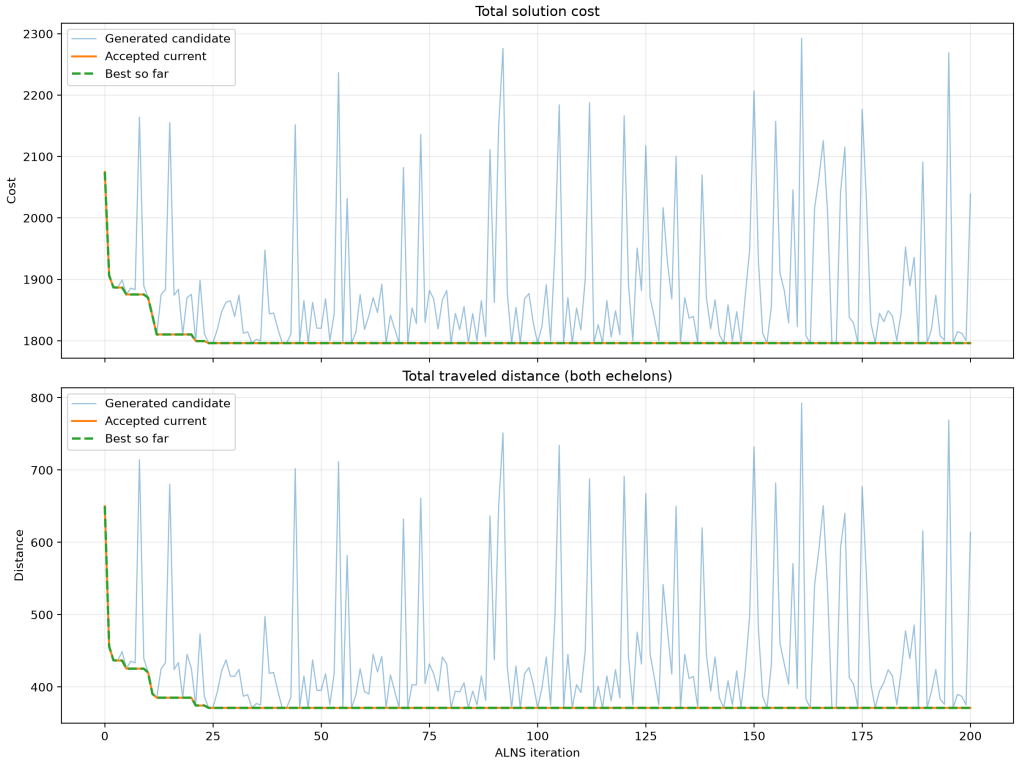


Figure 7: ALNS convergence for Random Insertion initialization without Simulated Annealing.

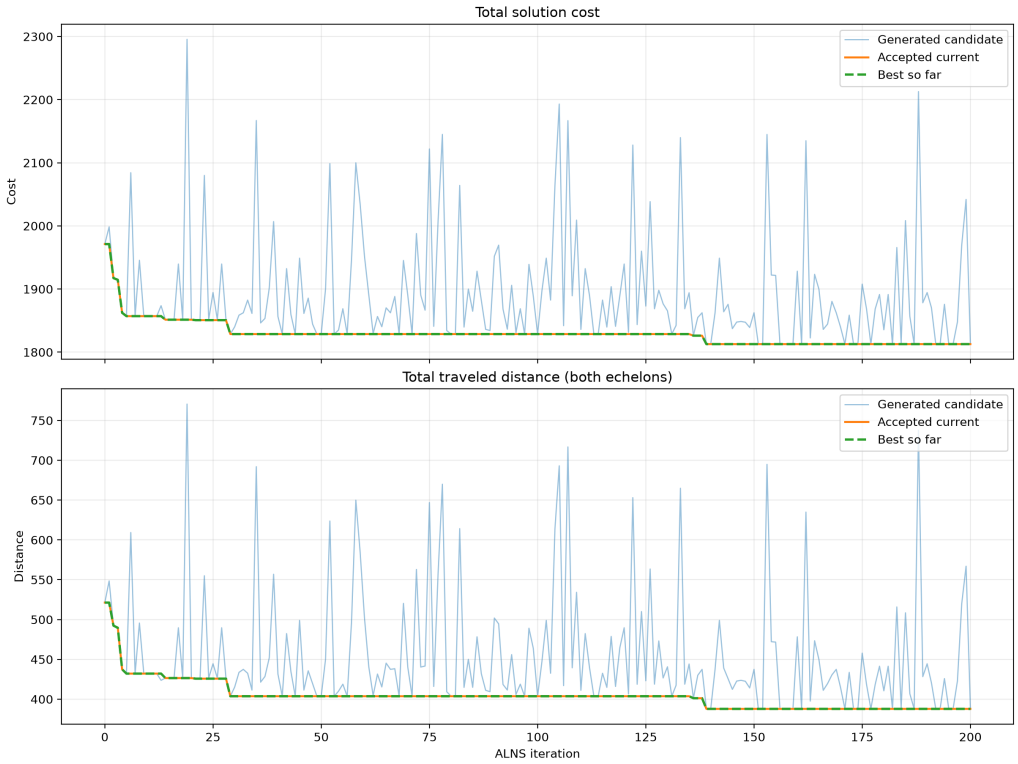


Figure 8: ALNS convergence for Clarke–Wright Savings initialization without Simulated Annealing.

H Complete ALNS Results With Simulated Annealing

Table 5 presents the complete results of SA-enabled ALNS for all 24 benchmark instances under both constructive heuristics, including initial and final costs, traveled distances, relative improvements, and computation times. ALNS improves all solutions, with mean cost reductions of 16.06% for Random Insertion and 8.16% for Clarke–Wright Savings, partly due to Random Insertion's higher initial costs. Clarke–Wright achieves lower final costs in 14 instances versus 10, indicating that its initial advantage narrows after ALNS.

Table 5: ALNS performance with Simulated Annealing (200 iterations, seed 42).

Instance	Constructive	Initial cost	Final cost	Initial dist.	Final dist.	Improv. (%)	Time (s)
Ca1-2,3,100	Random	12716.99	10777.23	3666.99	1702.23	15.25	11.99
Ca1-2,3,100	CW Savings	11318.82	10767.59	2193.82	1717.59	4.87	10.87
Ca1-2,3,15	Random	2074.93	1807.33	649.93	382.33	12.90	0.44
Ca1-2,3,15	CW Savings	1971.24	1799.27	521.24	374.27	8.72	0.42
Ca2-2,3,100	Random	13097.88	10988.33	3847.88	1813.33	16.11	12.12
Ca2-2,3,100	CW Savings	11614.96	10986.44	2339.96	1811.44	5.41	12.91
Ca3-3,5,50	Random	5543.85	4774.57	1493.85	724.57	13.88	4.31
Ca3-3,5,50	CW Savings	5186.25	4780.96	1036.25	730.96	7.81	4.20
Ca4-2,3,50	Random	6154.25	5338.92	1704.25	938.92	13.25	3.70
Ca4-2,3,50	CW Savings	5642.64	5329.17	1217.64	929.17	5.56	4.00
Ca5-6,4,50	Random	5846.63	4892.34	1671.63	717.34	16.32	4.06
Ca5-6,4,50	CW Savings	5264.19	4903.96	989.19	728.96	6.84	4.09
Cb1-2,3,50	Random	6033.67	4935.83	1933.67	885.83	18.20	3.86
Cb1-2,3,50	CW Savings	5331.35	4925.86	1206.35	875.86	7.61	3.57
Cb2-2,3,100	Random	11961.37	9998.11	3726.37	1713.11	16.41	12.00
Cb2-2,3,100	CW Savings	10583.71	9956.06	2273.71	1721.06	5.93	12.37
Cb3-3,5,50	Random	6351.72	5316.28	1811.72	826.28	16.30	4.06
Cb3-3,5,50	CW Savings	5817.25	5318.88	1202.25	828.88	8.57	4.14
Cb3-6,4,100	Random	12130.98	10038.57	3655.98	1563.57	17.25	12.53
Cb3-6,4,100	CW Savings	10851.34	10076.65	2326.34	1576.65	7.14	11.58
Cb4-3,5,100	Random	12499.03	10499.98	3534.03	1634.98	15.99	12.35
Cb4-3,5,100	CW Savings	11465.17	10542.55	2425.17	1677.55	8.05	13.51
Cb5-3,5,100	Random	12118.71	10532.03	3233.71	1672.03	13.09	12.23
Cb5-3,5,100	CW Savings	11503.96	10551.53	2568.96	1716.53	8.28	12.20
Cc1-2,3,50	Random	6196.23	5008.02	2021.23	883.02	19.18	4.09
Cc1-2,3,50	CW Savings	5349.63	4987.07	1149.63	862.07	6.78	3.68
Cc1-6,4,50	Random	6114.40	5043.07	1839.40	768.07	17.52	4.05
Cc1-6,4,50	CW Savings	5597.36	5020.46	1147.36	745.46	10.31	4.10
Cc2-6,4,15	Random	2119.04	1837.02	694.04	362.02	13.31	0.55
Cc2-6,4,15	CW Savings	2021.91	1833.90	521.91	358.90	9.30	0.54
Cc3-3,5,50	Random	6268.65	5018.94	1893.65	743.94	19.94	4.08
Cc3-3,5,50	CW Savings	5476.91	5045.59	1101.91	770.59	7.88	4.00
Cc4-6,4,15	Random	2099.76	1782.31	599.76	282.31	15.12	0.54
Cc4-6,4,15	CW Savings	2086.50	1777.95	486.50	277.95	14.79	0.53
Cc5-6,4,100	Random	12569.12	10175.39	3969.12	1575.39	19.04	12.20
Cc5-6,4,100	CW Savings	11080.54	10153.60	2430.54	1553.60	8.37	11.78
Cd1-6,4,15	Random	2156.18	1693.87	731.18	318.87	21.44	0.50
Cd1-6,4,15	CW Savings	2116.68	1694.66	591.68	319.66	19.94	0.50
Cd2-6,4,100	Random	12036.79	10107.47	3436.79	1507.47	16.03	12.31
Cd2-6,4,100	CW Savings	10733.99	10048.65	2083.99	1448.65	6.38	11.63

Continued on next page

Table 5 continued from previous page

Instance	Constructive	Initial cost	Final cost	Initial dist.	Final dist.	Improv. (%)	Time (s)
Cd2-6,4,50	Random	6011.80	5045.50	1686.80	820.50	16.07	3.99
Cd2-6,4,50	CW Savings	5445.73	5055.38	1145.73	830.38	7.17	4.08
Cd3-6,4,50	Random	6496.05	5404.63	1896.05	804.63	16.80	4.06
Cd3-6,4,50	CW Savings	5897.50	5390.36	1197.50	790.36	8.60	4.16
Cd4-2,3,100	Random	12060.67	10349.28	3560.67	1899.28	14.19	11.40
Cd4-2,3,100	CW Savings	10728.26	10372.02	2253.26	1922.02	3.32	11.87
Cd5-3,5,100	Random	11736.62	10354.54	3086.62	1679.54	11.78	12.64
Cd5-3,5,100	CW Savings	11230.58	10307.36	2480.58	1657.36	8.22	12.20

I ALNS Convergence with Simulated Annealing

The following figures show candidate, accepted-current, and best-so-far costs and distances for the SA-enabled runs on instance `Ca1-2,3,15`.



Figure 9: SA-enabled ALNS convergence using Random Insertion initialization.

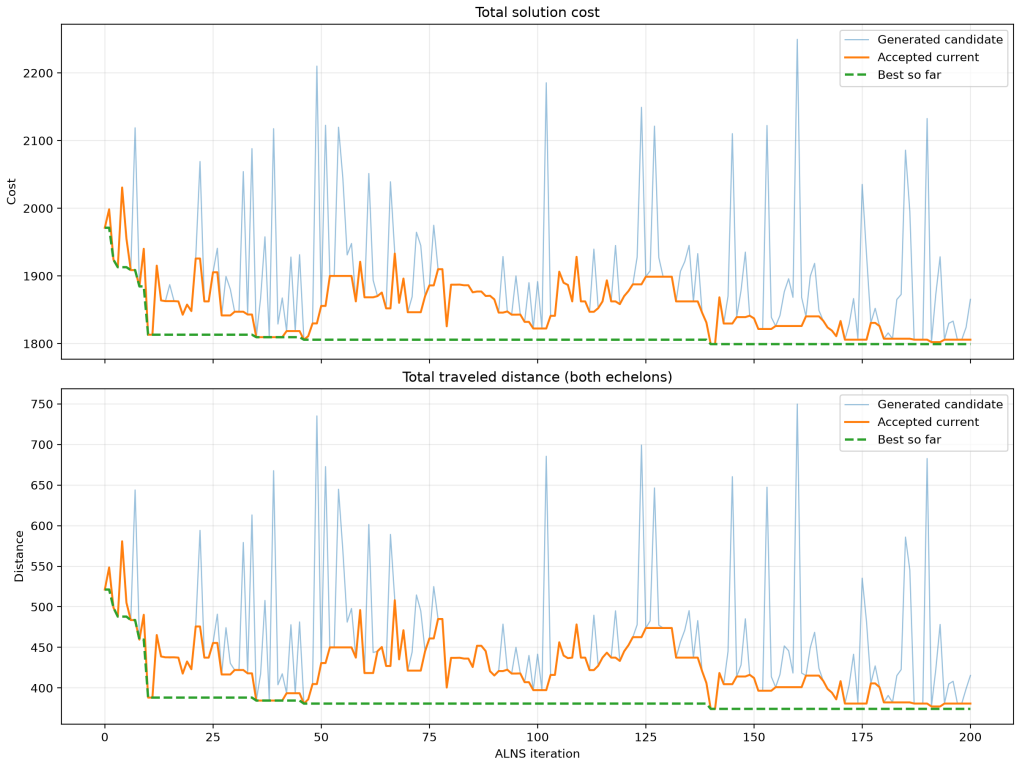


Figure 10: SA-enabled ALNS convergence using Clarke–Wright Savings initialization.

J Comparison of Acceptance Strategies

Figure 11 shows the percentage difference in best-found solution costs between ALNS with and without Simulated Annealing for all 24 benchmark instances under both constructive heuristics. Negative values indicate better performance with SA, while positive values indicate better performance without SA.

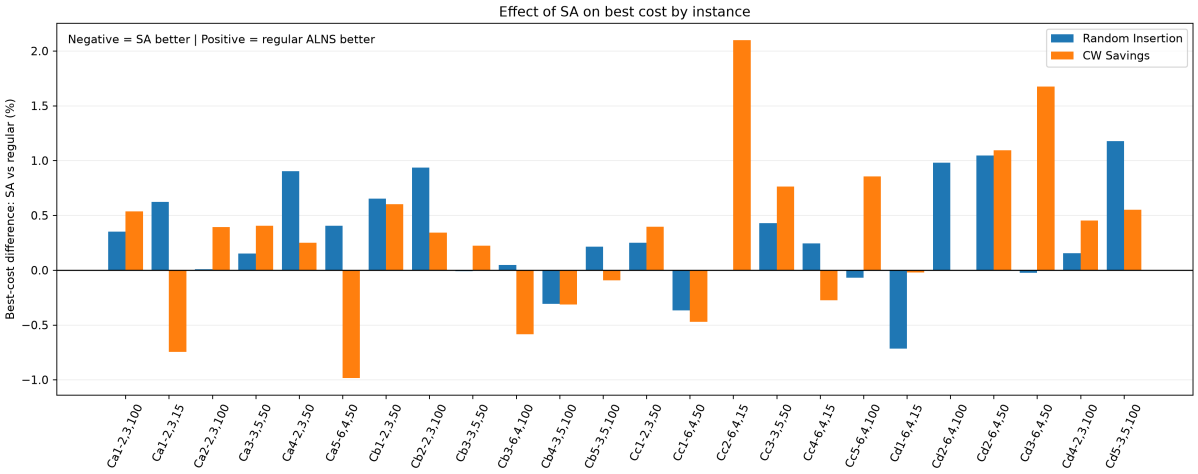


Figure 11: Percentage difference in best-found solution cost between ALNS with and without Simulated Annealing across all 24 instances and both constructive heuristics.